

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

INF01120 – Desenvolvimento de Software

Trabalho Prático – Fase 2

Integrantes:

Guilherme Souza da Roza Lobato (00584649)

Cristiano Zandoná Parnoff (00584548)

Observação: documento consolidado a partir da documentação produzida nas Fases 1 e 2.

1. Introdução

O objetivo do projeto é desenvolver um Gerador de Música a partir de Texto capaz de interpretar caracteres, transformá-los em eventos musicais e reproduzir o resultado utilizando instrumentos MIDI. O foco principal do trabalho foi aplicar conceitos de Engenharia de Software, boas práticas de programação, modularidade, Orientação a Objetos, princípios SOLID e evolução contínua do software diante das mudanças de requisitos.

2. Requisitos Funcionais – Fase 1

- Ler texto de entrada através da interface.
- Interpretar caractere por caractere aplicando as regras do enunciado.
- Gerar saída sonora reproduzível.
- Permitir configuração de BPM, instrumento inicial, oitava padrão e volume.
- Mapear A-G para notas e H para Si Bemol.
- Tratar letras minúsculas como pausas.
- Aplicar trocas de instrumentos conforme caracteres especiais.
- Alterar volume através de espaços.
- Alterar oitava através de ? e .
- Exibir visualmente a sequência gerada.
- Permitir parar, limpar e reproduzir novamente.

3. Requisitos Não Funcionais – Fase 1

- Modularidade.
- Baixo acoplamento e alta coesão.
- Aplicação dos princípios SOLID.
- Legibilidade do código.

- Ausência de duplicação.
- Facilidade de manutenção.
- Testabilidade.
- Utilização de GitHub para versionamento.

4. Requisitos Adicionados – Fase 2

- Importação de arquivos TXT.
- Salvamento de arquivos TXT.
- Exportação de arquivos MIDI.
- Utilização obrigatória do padrão General MIDI.
- Processamento de múltiplas vozes.
- Polifonia inspirada em fugas de Bach.
- Delays por voz através da sintaxe [n].
- Oitava base específica por voz.
- Volume base específico por voz.
- Instrumento base específico por voz.
- Controle global de BPM através dos caracteres > e <.
- Reprodução simultânea das vozes.

5. Definição das Classes – Fase 1

Classe EstadoMusical

Atributos: instrumento, oitava, volume, ultima_nota, anterior_era_nota.

Métodos: dobrar_volume(), subir_oitava(), trocar_instrumento(), somar_instrumento(), registrar_nota().

Classe EventoMusical

Atributos: tipo, char, nota, oitava, volume e instrumento.

Classe RegraBase

Métodos: deve_processar() e processar().

Classe MusicService/PolifoniaService

Responsável pela geração da sequência musical e aplicação das regras.

6. Definição das Classes – Fase 2

Classe Voz

Representa uma linha independente da composição.

Atributos: id_voz, texto_original, atraso_beats e estado.

Métodos: inicializar_atributos_base() e extrair_atraso().

Classe ContextoGlobal

Responsável pelo BPM global.

Métodos: acelerar() e desacelerar().

Classe EventoMusical (Atualizada)

Novos atributos: voz_id e beat_absoluto.

Classe PolifoniaService

Responsável pela criação da timeline global e sincronização das vozes.

Classe MidiExporter

Responsável pela geração do arquivo MIDI utilizando MidiUtil.

7. Arquitetura do Sistema

O sistema foi dividido em Backend e Frontend.

Backend:

- Python.
- FastAPI.
- MidiUtil.
- Regras de negócio.
- Geração de timeline.
- Exportação MIDI.

Frontend:

- React.
- TypeScript.
- Vite.
- TailwindCSS.
- Tone.js.
- Reprodução sonora.
- Visualização da sequência musical.

A arquitetura cliente-servidor foi escolhida para separar responsabilidades e facilitar a manutenção.

8. Fluxo Completo do Sistema

1. Usuário digita texto ou importa um TXT.
2. Frontend coleta texto e configurações.
3. Dados são enviados via API.
4. Backend valida a entrada.

5. Texto é separado em vozes.
6. Cada voz recebe seus atributos iniciais.
7. Caracteres são processados pelas regras.
8. Eventos musicais são criados.
9. Todos os eventos são inseridos em uma timeline global.
10. BPM global é propagado.
11. MidiExporter gera o arquivo MIDI.
12. Backend retorna timeline e MIDI.
13. Frontend reproduz a música.
14. Usuário pode baixar TXT ou MIDI.

9. Evolução Arquitetural da Fase 2

A principal mudança foi a transição de uma sequência musical linear para uma estrutura polifônica.

Foi criada a abstração de Voz, permitindo que cada linha do texto fosse processada de maneira independente.

Também foi criada uma timeline global capaz de sincronizar eventos de diferentes vozes e um exportador MIDI capaz de gerar múltiplas trilhas.

10. Refatoração do Backend

Foi realizada a eliminação de bad smells relacionados ao processamento de notas compostas.

Problema:

O Mi Bemol (Mb) era tratado através de substituições artificiais na string.

Solução:

O algoritmo foi alterado para utilizar lookahead.

Quando o sistema encontra M, verifica se o próximo caractere é b.

Caso positivo, processa ambos como Mb.

Isso eliminou acoplamentos indevidos e tornou o processamento mais robusto.

11. Implementação do MidiExporter

Foi criada a classe MidiExporter utilizando a biblioteca MidiUtil.

Responsabilidades:

- Criar múltiplas trilhas MIDI.
- Configurar canais por voz.
- Configurar instrumentos.

- Inserir notas respeitando beat_absoluto.
- Gerar arquivo MIDI.
- Retornar o resultado em Base64 para o frontend.

12. Alterações do Frontend

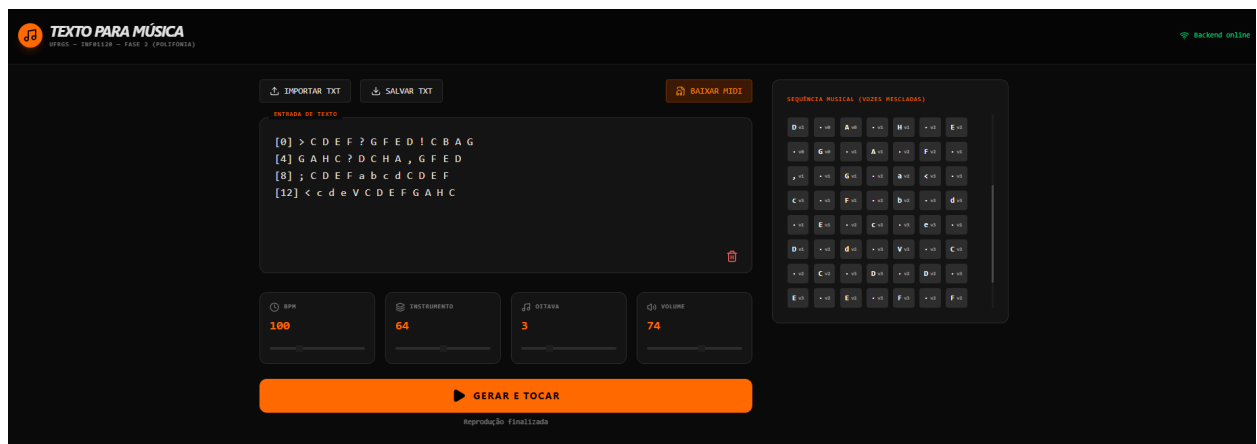
Foram adicionadas funcionalidades para:

- Importar arquivos TXT.
- Salvar arquivos TXT.
- Baixar arquivos MIDI.
- Visualizar eventos mesclados.
- Reproduzir múltiplas vozes simultaneamente.
- Exibir destaque visual durante a execução.

A lógica de reprodução foi migrada para um modelo baseado em timeline absoluta.

13. Interface do Usuário

Descrição da interface implementada e justificativas do layout.



A interface foi organizada para priorizar a inserção do texto. Os controles de BPM, instrumento, oitava e volume ficam agrupados abaixo da área principal. Os botões de importação, exportação TXT e download MIDI ficam em destaque. O painel lateral apresenta a sequência musical gerada.

14. Procedimento de Testes

- Teste 1 – Mapeamento de notas.
- Teste 2 – Alteração de instrumentos.
- Teste 3 – Alteração de volume.
- Teste 4 – Alteração de oitava.

Teste 5 – Controle global de BPM.
Teste 6 – Processamento de múltiplas vozes.
Teste 7 – Delays utilizando [n].
Teste 8 – Exportação MIDI.
Teste 9 – Importação TXT.
Teste 10 – Exportação TXT.

Todos os testes foram executados manualmente durante o desenvolvimento e utilizados para validar o comportamento esperado.

15. Suposições Adotadas

- Volume máximo igual a 127.
- BPM inicial configurável.
- Canal MIDI 9 não utilizado.
- Reutilização cíclica dos parâmetros das vozes.
- Mb representa Mi Bemol.
- Downloads utilizam o comportamento padrão do navegador.
- Instrumentos seguem o padrão General MIDI.

16. Aplicação dos Princípios SOLID

SRP: responsabilidades separadas entre interpretação, exportação e reprodução.

OCP: novas regras podem ser adicionadas através de novas classes derivadas de RegraBase.

LSP: regras especializadas substituem a abstração base sem alterar comportamento esperado.

ISP: interfaces específicas para cada responsabilidade.

DIP: dependência de abstrações em vez de implementações concretas.

17. Conclusão

O projeto evoluiu significativamente entre as Fases 1 e 2. A arquitetura modular permitiu incorporar

polifonia, múltiplas vozes, exportação MIDI e manipulação de arquivos sem comprometer a estrutura original.

A aplicação dos princípios de Engenharia de Software contribuiu para a extensibilidade e

manutenção do sistema.

18. Link do GitHub

<https://github.com/CrisParnoff/DS-INF01120>